

that we are trying to design. With this, I'll highlight the functions libPCAP will need to perform.

First, find out the device you want to sniff on. In Linux, this can be *eth0* (ethernet), or *wlan0* (wireless). This may vary based on your network connection. LibPCAP can automatically find which device you want to sniff on. Next, set up PCAP and initialise it, to sniff on the device you found or set up in the previous step. You can then set up filters, to capture only the kind of traffic you are interested in. When you specify a filter, then you must compile it, for correctness, before you apply it. Finally, make libPCAP grab packets in a loop, until your needs are satisfied.

Opening the device for sniffing

It is remarkably simple to set up your device. If you know your device string (*eth0*, *wlan0*, etc), you can set it directly, or pass it as a parameter to your program. If you want libPCAP to find your default device, use *pcap_lookupdev* (*char *errbuf*). The parameter *errbuf* is buffer memory (array), which stores the description of the error in case the command fails. Most libPCAP commands let you pass this array as a parameter, and it universally performs the same function.

The code to find out the default device is given below:

```
#include<stdio.h>
#include<pcap.h>
int main()
{
    char *dev,errbuf[PCAP_ERRBUF_SIZE];
    dev=pcap_lookupdev(errbuf);
    printf("Device: %s\n",dev);
    return 0;
}
```

The size of the error buffer *errbuf* is defined to be *PCAP_ERRBUF_SIZE*. This is a value predefined in the *pcap.h* header file. Compile the above code with the following command:

```
gcc code1.c -lpcap
```

-lpcap is the linker directive to link the associated libPCAP shared library, and must be used every time you want to compile code that uses libPCAP. You may also need to run your compiled program with root privileges.

Alternatively, if you know the device you want to sniff on, you can simply call the function:

```
pcap_lookup with the dev = " device id for the device you
want to sniff on" eg - dev="wlan0"
```

To open the device for sniffing, use:

```
pcap_t * pcap_open_live (char *device,int snaplen,int
promisc,int to_ms,char *errbuf)
```

The values of the parameters in the above defined function are as follows.

- *device*: This argument stores the device specified above (the device to be sniffed on).
- *snaplen*: This is an integer that defines the maximum number of bytes to be captured.
- *Promisc*: This is an integer that sets the device to work in promiscuous mode. Promiscuous mode sniffs all traffic on the wire – this could be all the traffic on the network. The downside of using promiscuous mode is that it can be easily detected, and the host may be taxed for resources. In non-promiscuous mode, the sniffer only captures packets that are relevant to the host, which means that only traffic that is to, from or routed by the host will be captured.
- *to_ms*: This is an integer that specifies the read time out in milliseconds (a value of 0 specifies no time out).
- *errbuf*: This stores the error message in case the call fails.

Notice that the function returns a pointer value of type *pcap_t*. This function returns a value that is your session handler. The code to open the connection is given below:

```
#include<stdio.h>
#include<pcap.h>
int main()
{
    char *dev,errbuf[PCAP_ERRBUF_SIZE];
    dev=pcap_lookupdev(errbuf);
    pcap_t *handle;
    handle=pcap_open_live(dev,BUFSIZ,1,0,errbuf);
    return 0;
}
```

The value of *BUFSIZ* is defined in *pcap.h*. In the above code, you are passing the value of the device you want to sniff on, which was found by the *pcap_lookupdev* function, while you are setting the maximum number of bytes to be captured to be *BUFSIZ*, the packet sniffing mode to be promiscuous, and no time out on read time.

Filtering traffic

At times, you will be interested in only certain types of traffic, rather than in capturing all packets. Here is where filters come in. They let you only capture certain kinds of traffic. For example, you may only want packets from a specific protocol like UDP or TCP. Now, one may wonder why you shouldn't use *if/else* statements? Well, you could, but filters are easier to use, and are more efficient. Before you apply your filter, you must compile it. The expression for the filter is a string. Compile it with the following function:

```
int pcap_compile(pcap_t *handler,struct bpf_program
*filter,char *flt,int opt, bpf_u_int32 netmask)
```